



THE UNIVERSITY
OF QUEENSLAND
A U S T R A L I A

Inferring security-relevant properties from Java documentation

Connor Myers

Bachelor of Engineers (Honours)

Abstract

Start this section on a new page [this template will automatically handle this].

The abstract should outline the main approach and findings of the thesis and normally must be between 300 and 800 words.

Declaration by author

(All candidates to reproduce this section in their thesis verbatim)

This thesis is composed of my original work, and contains no material previously published or written by another person except where due reference has been made in the text. I have clearly stated the contribution by others to jointly-authored works that I have included in my thesis.

I have clearly stated the contribution of others to my thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, financial support and any other original research work used or reported in my thesis. The content of my thesis is the result of work I have carried out since the commencement of my higher degree by research candidature and does not include a substantial part of work that has been submitted to qualify for the award of any other degree or diploma in any university or other tertiary institution. I have clearly stated which parts of my thesis, if any, have been submitted to qualify for another award.

I acknowledge that an electronic copy of my thesis must be lodged with the University Library and, subject to the policy and procedures of The University of Queensland, the thesis be made available for research and study in accordance with the Copyright Act 1968 unless a period of embargo has been approved by the Dean of the Graduate School.

I acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate I have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors for any jointly authored works included in the thesis.

Chapter 1

Introduction

Static application security testing (SAST) applies static program analysis to the security of software projects. Static analysis is performed on the source code, compiled byte-code or some other intermediate representation of the source code [1].

SAST tools have become a widespread method for improving the security of large software projects. Popular SAST tools include: SonarQube [2], SpotBugs [3], Snyk Code [4], and Facebook's Infer [5]. SAST tools have become commonly used in organisations at all software development life cycle stages. They have been demonstrated to be highly effective in identifying defects in programs compared to other methods, such as manual inspection [6].

SAST tools, by their very nature, produce a conservative program analysis. As a result of this, false positives are a common issue with these tools. False positives are not ideal as manual inspection is required to identify the genuine defects from these false positives, leading to increased time and labour cost, which are the same measures SAST tools aim to minimise. Manual configuration of SAST tools is often required to reduce the rate of false positives and produce a more effective and useful analysis for specific software projects [7].

1.1 Topic Definition

Introduce your topic.

Chapter 2

Full title

Introduce the broad layout of the chapter.

2.1 Introduction

Add your text here.

Chapter 3

Conclusion

Conclude your thesis.

Bibliography

- [1] F. Nielson, H. R. Nielson, C. Hankin, Principles of program analysis / Flemming Nielson, Hanne Riis Nielson, Chris Hankin., 1st Edition, Springer-Verlag, Berlin ; Heidelberg ; New York, 2005.
- [2] SonarSource, Sonarqube (2022).
URL <https://www.sonarqube.org/>
- [3] Spotbugs (2022).
URL <https://spotbugs.github.io/>
- [4] Snyk, Snykcode (2022).
URL <https://snyk.io/>
- [5] Facebook, Infer (2022).
URL <https://fbinfer.com/>
- [6] W. R. Nichols, The cost and benefits of static analysis during development (2020). doi:10.48550/ARXIV.2003.03001.
URL <https://arxiv.org/abs/2003.03001>
- [7] F. Zampetti, S. Scalabrino, R. Oliveto, G. Canfora, M. Di Penta, How open source projects use static code analysis tools in continuous integration pipelines, in: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), IEEE, 2017, pp. 334–344.

Appendix A

Appendix

Write your appendix here.